Lecture 4

Tree method

The Recurrence Blueprint

Deconstructing T(n) before visualization.

The Node Cost
The actual work done at this level (e.g., cn). This is what we sum.

$$T(n) = aT(n/b) + f(n)$$

Branching Factor
Determines tree width (children per node).

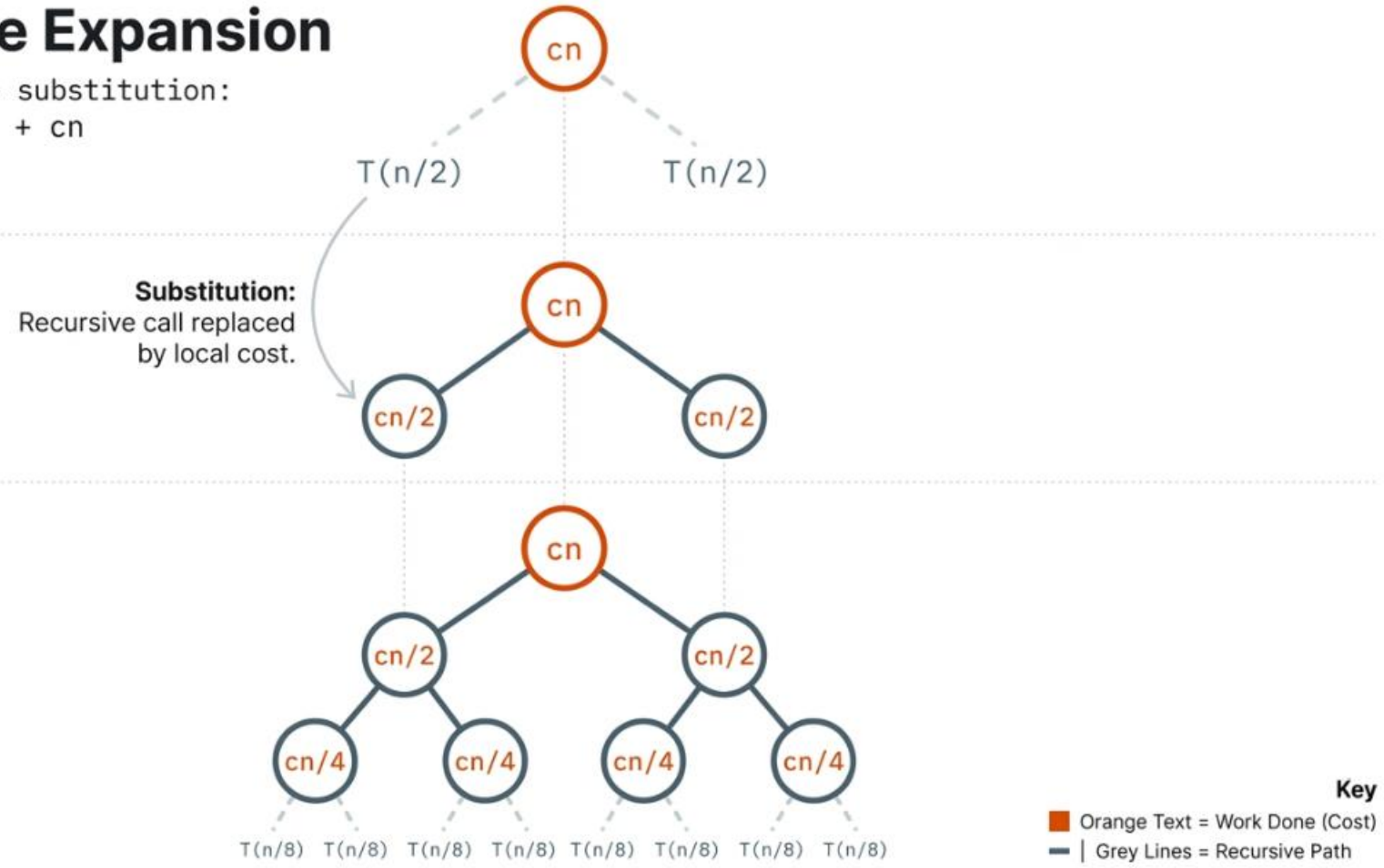
Sub-problem Size
Determines tree depth (how fast input shrinks).

OBJECTIVE: Convert this algebraic relation into a geometric map, then sum the cost of every node to find the Total Time.

© 2026 Ahmed Elrefaiy

Recursive Expansion

Visualizing the substitution:
$T(n) = 2T(n/2) + cn$

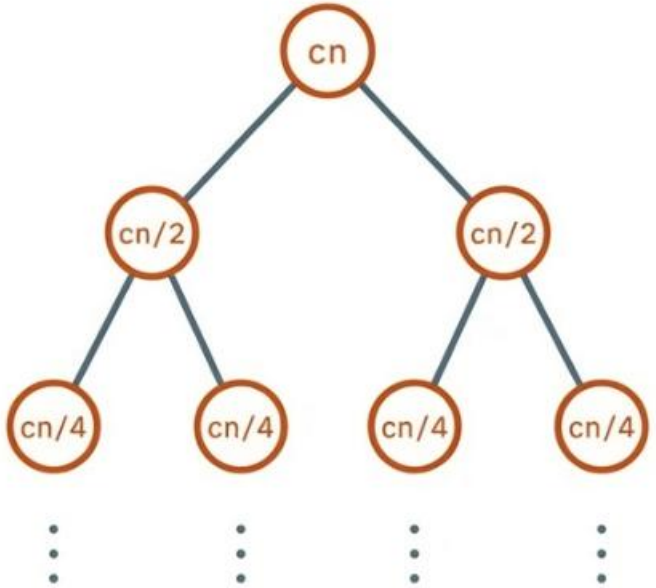


Analyzing Dimensions

Calculating height and level costs.

Height: $h = \log_2 n$

Input divides by 2 at each step.



Sum = cn

$2 \times (cn/2) = cn$

$4 \times (cn/4) = cn$

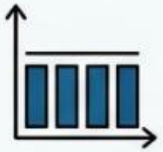
PATTERN OBSERVATION:
Despite the tree getting wider, the total work at each level remains constant (cn).

Summation Logic

How to add the levels based on their relationship

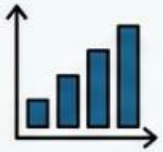
Common Summation Patterns

Constant Series



$$\text{Sum} = \frac{n - n}{n}$$

Arithmetic Series



$$\text{Sum} = n(n+1)(2n+1)$$

Quadratic Series

(The Square Case)

$$\text{Sum} = \frac{n(n+1)(2n+1)}{6}$$

Geometric Series

Decreasing ($r < 1$, Root Dominates):

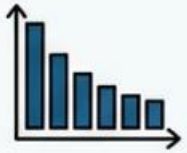
$$\frac{a}{1-r}$$

Increasing ($r > 1$, Leaves Dominate):

$$a\left(\frac{(1-r^h)}{1-r}\right)$$

Harmonic Series

(The Reciprocal Case)



$$\text{Sum} \approx \ln(n)$$

Final Synthesis

Deriving the Big O Complexity.

Total Time $T(n) = \sum$ (Level Costs)

↓

Since cost is constant (cn) across all levels:

↓

$T(n) = cn \times (\text{Height})$

↓

$T(n) = cn \times \log_2 n$

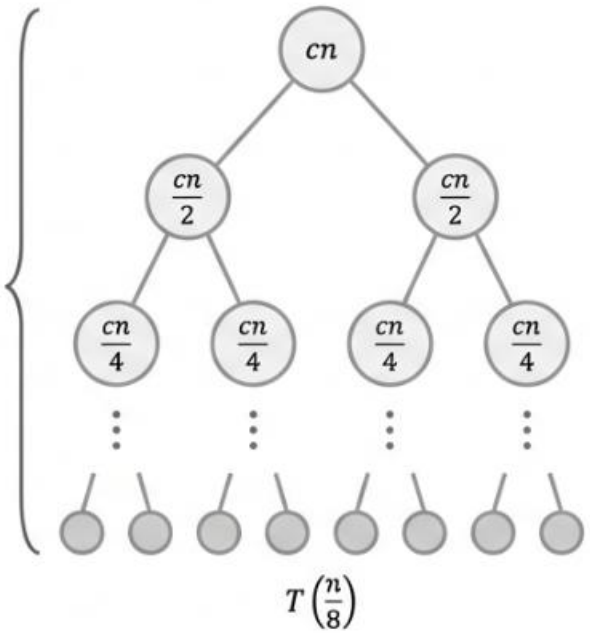
$$O(n \log n)$$

Constant Series (The Equal Case)

Equal Costs: Level 0 = Level 1 = Level 2...
Total = Cost × Height
$cn \times \log n$

Height: $h = \log_2 n$

Input divides by 2 at each step.



Sum = cn

$2 \times \left(\frac{cn}{2}\right) = cn$

$4 \times \left(\frac{cn}{4}\right) = cn$

PATTERN OBSERVATION:
Despite the tree getting wider, the total work at each level remains constant (cn).

Analyst Note:
Always check the 'leaves' vs the 'root'. If the series is decreasing ($r < 1$), the recursion fades out—the cost is dominated by the root. If increasing, the cost is dominated by the leaves.

The Expanding Cost

$$T(n) = 2T(n/2) + 10$$

SPECIES: GEOMETRIC INCREASING

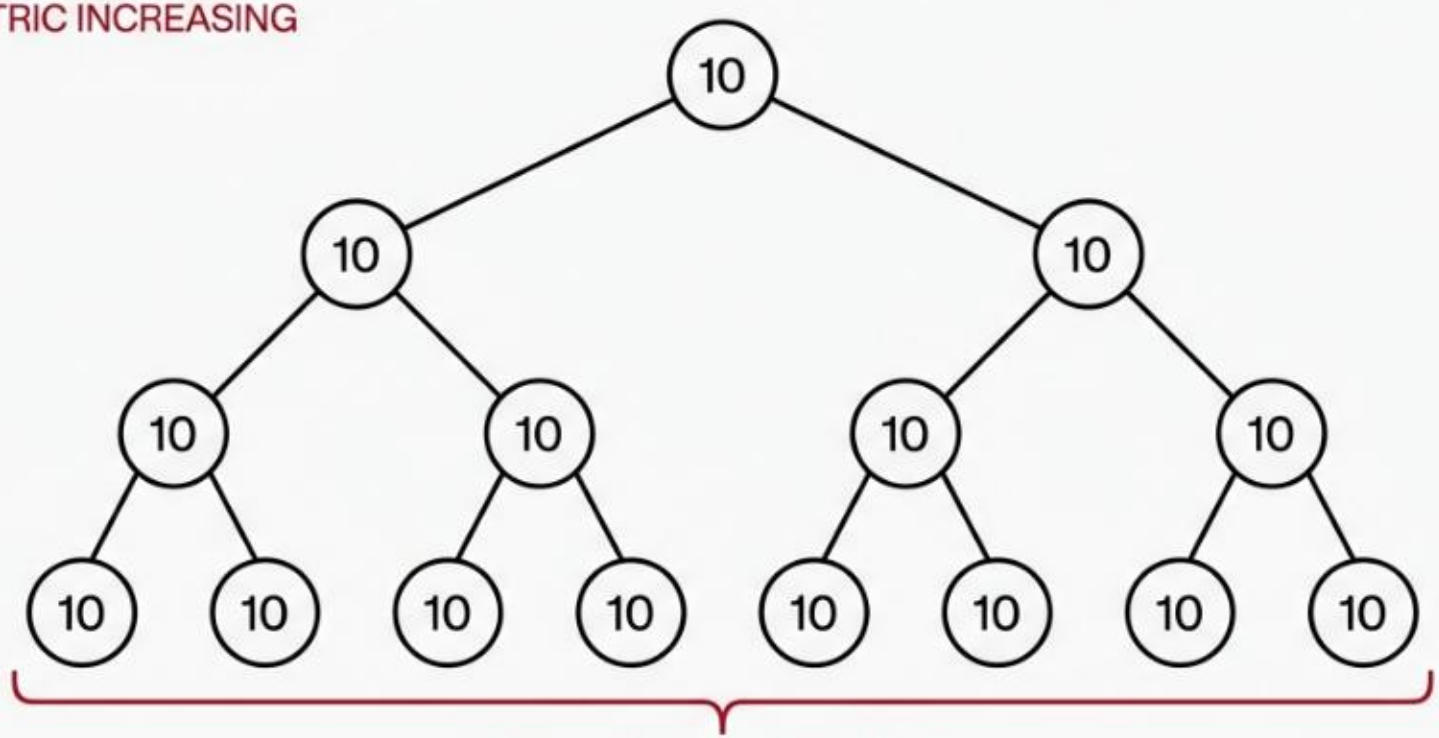


Exhibit Label

Tree Depth: log_2 n

Series: 10, 20, 40, 80...

Complexity: Θ(n)

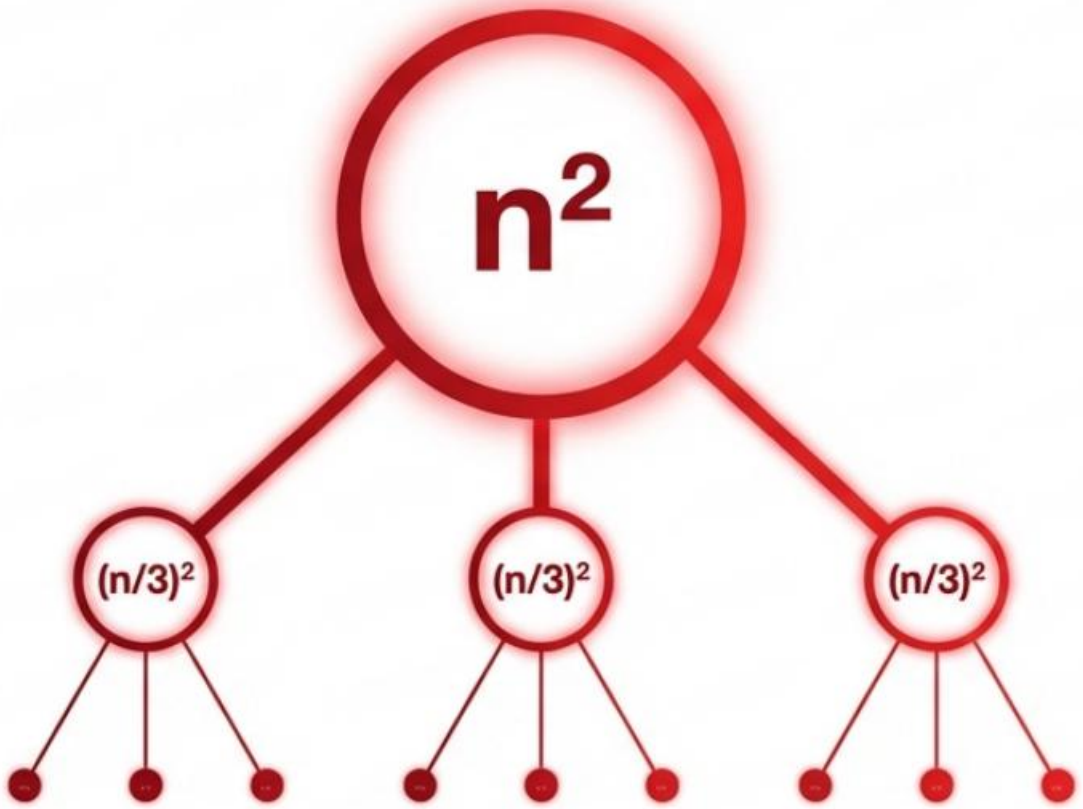
Applied Role

Context: A traversal algorithm (like counting nodes) where the visit cost is constant, but the sheer volume of nodes doubles at every step.

The Rapid Shrinkage

Species: Geometric Decreasing

$$T(n) = 3T(n/3) + n^2$$



Key Insight

Root Dominated. The quadratic work at the start is so heavy it dwarfs all subsequent recursive calls.

EXHIBIT LABEL

Tree Depth: $\log_3 n$
Series: $n^2 + 1/3\, n^2 + 1/9\, n^2 \ldots$
Complexity: $\Theta(n^2)$

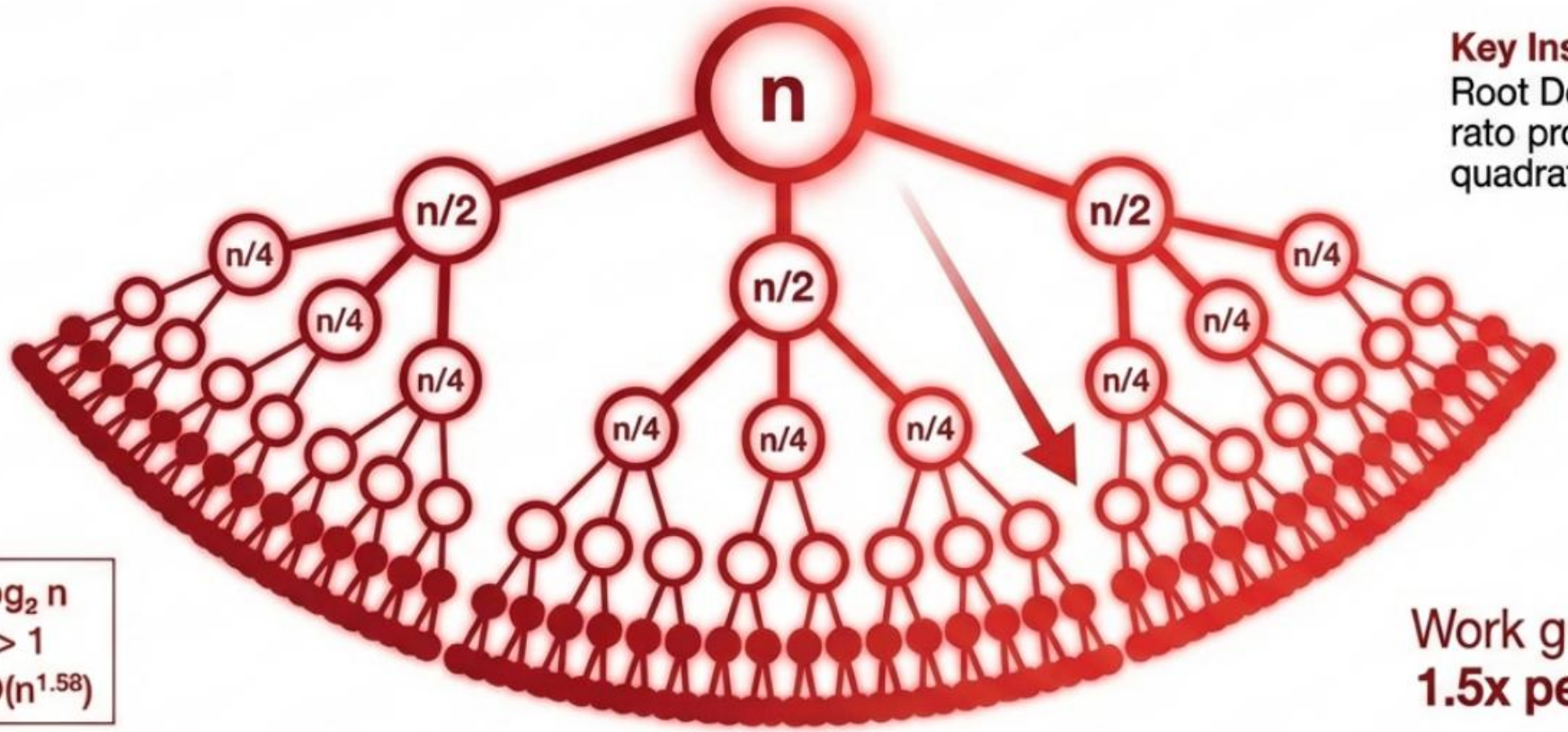
Applied Role

Context: Algorithms with an expensive ‘partition’ or ‘combine’ step that acts as the primary bottleneck.

The Increasing Workload

Species: Geometric Increasing (Heavy Branching)

$$T(n) = 3T(n/2) + n$$



Key Insight
Root Dominated. The rato problems beats quadratic work it.

EXHIBIT LABEL

Tree Depth: $\log_2 n$
Ratio: $r = 1.5 > 1$
Complexity: $\Theta(n^{1.58})$

Work grows by **1.5x per level.**

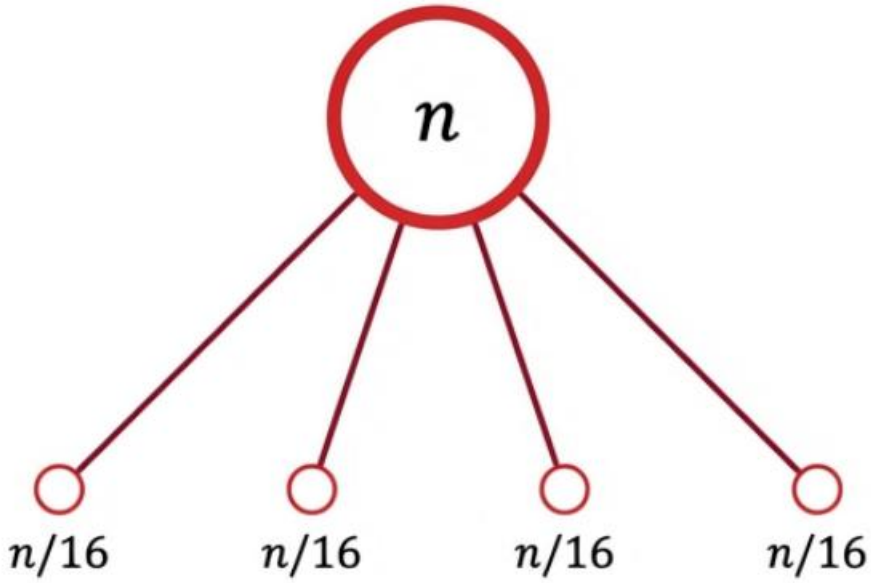
Applied Role

Context: Karatsuba Multiplication. Splitting into 3 sub-problems beats the quadratic naive approach (n^2) but is costlier than linear sorting ($n \log n$).

The Aggressive Decay

Species: Geometric Decreasing (Fast Reduction)

$$T(n) = 4T(n/16) + n$$



Narrative Text

Even with 4 branches, dividing the problem size by 16 shrinks the workload so fast that the total time is proportional only to the first step.

EXHIBIT LABEL

Tree Depth: $\log_{16} n$

Series: $n + 0.25n + \ldots$

Complexity: $\Theta(n)$

Applied Role: Context: Highly efficient divide-and-conquer where sub-problems are a tiny fraction of the original input.

The Unbalanced Split

Species: Geometric Asymmetric

$$T(n) = T(n/3) + T(2n/3) + cn$$

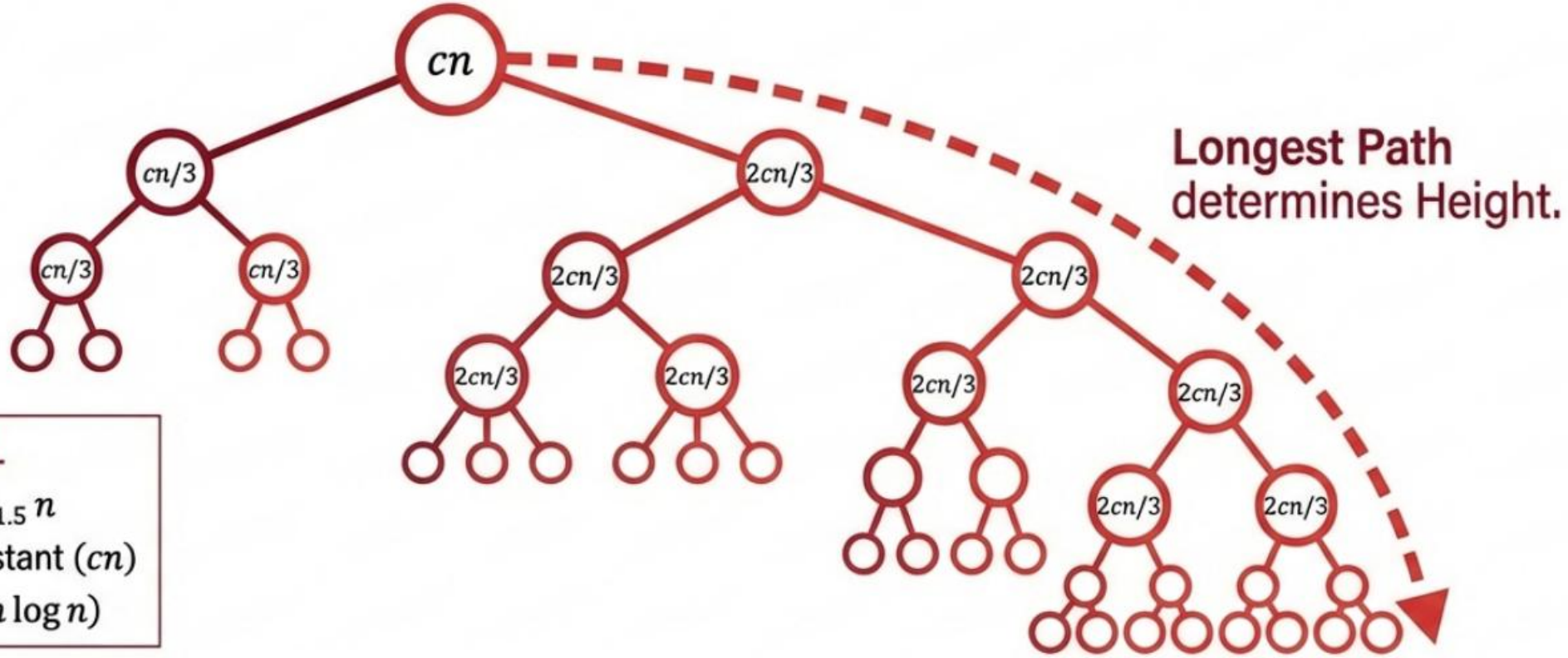


EXHIBIT LABEL

Tree Height: $\log_{1.5} n$

Level Cost: Constant (cn)

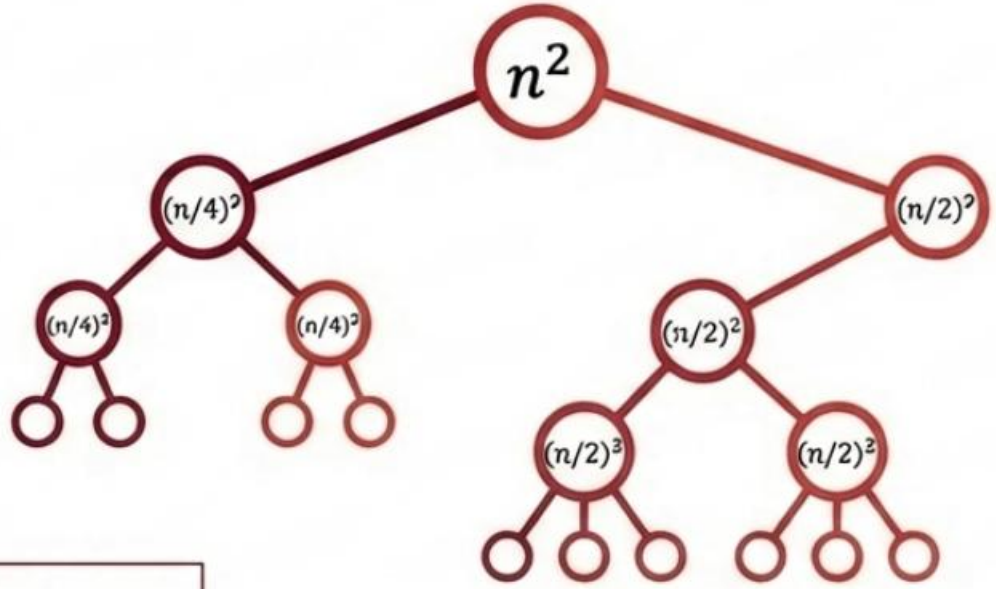
Complexity: $\Theta(n \log n)$

Applied Role: Context: Quicksort with a biased pivot. The recursion depth is uneven, but the total work per level remains constant.

The Multi-Part Cost

Species: Geometric Uneven

$$T(n) = T(n/4) + T(n/2) + n^2$$



Level 1 Cost:

$$\left(\frac{1}{4}\right)^2 + \left(\frac{1}{2}\right)^2$$

$$= \frac{1}{16} + \frac{4}{16}$$

$$= \frac{5}{16}$$

$\frac{5}{16} < 1$ (Decaying Series)

EXHIBIT LABEL

Dominant Term: The Root

Complexity: $\Theta(n^2)$

Applied Role: Context: Complex recursive functions where the combination step involves quadratic work, overpowering the sub-problems.

The Linear Chain

Species: Constant Arithmetic

$$T(n) = T(n-1) + 1$$

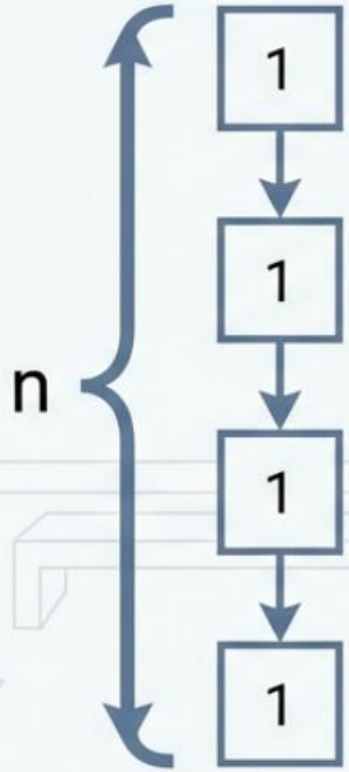


EXHIBIT LABEL

Tree Depth: n
Summation: Sum of 1, n times
Complexity: Θ(n)

Key Insight

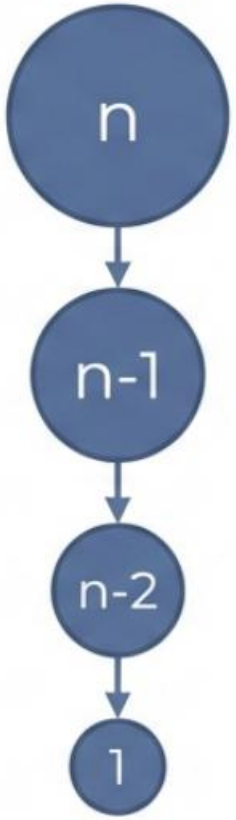
- Recursion by subtraction creates a deep line, not a wide tree.
- This is functionally identical to a simple "for" loop.

Applied Role **Context:** Linear search or traversing a linked list recursively.

The Linear Arithmetic

Species: Arithmetic Increasing

$$T(n) = T(n-1) + n$$



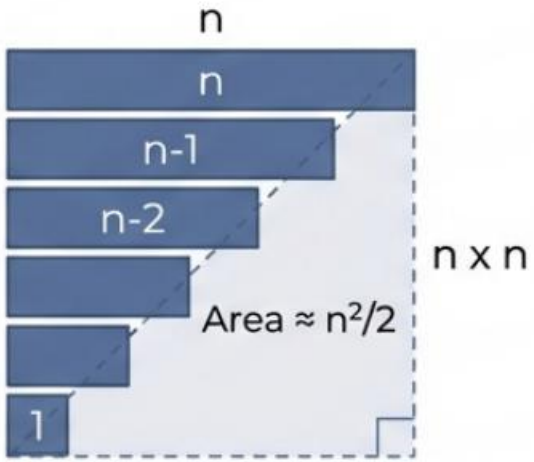


EXHIBIT LABEL

Series: Sum of integers 1 to n

Formula: n(n+1) / 2

Complexity: Θ(n^2)

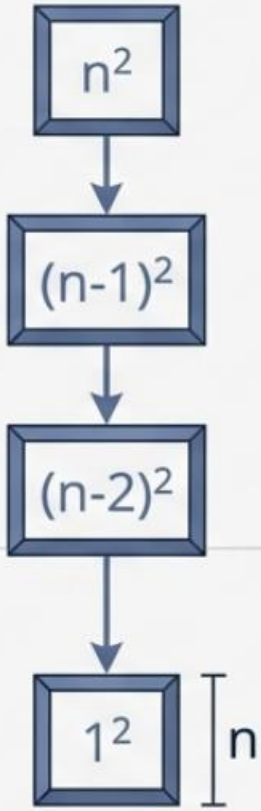
Applied Role

Context: Worst-case Insertion Sort or Selection Sort. The work shrinks linearly, but the sum creates a quadratic runtime.

The Quadratic Arithmetic

Species: Cubic Summation

$$T(n) = T(n-1) + n^2$$



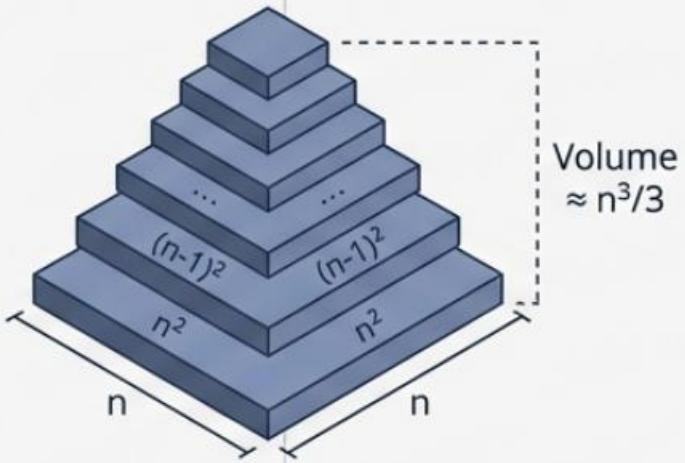


EXHIBIT LABEL

Series: Sum of squares
Dominant Term: n^3
Complexity: $\Theta(n^3)$

Key Insight

If the work per step is quadratic and the depth is linear, the total complexity becomes cubic.

Applied Role

Context: A recursive function containing a nested loop at every step.

The Step-Skipper

Species: Arithmetic (Step > 1)

$$T(n) = T(n-2) + n$$

Reducing the height by a factor of 2 (dividing by constant) does not change the Big-O complexity class. It remains quadratic.

EXHIBIT LABEL

- Series: Sum of even/odd numbers
- Calculation: ~ (n/2 * n) / 2
- Complexity: Θ(n^2)

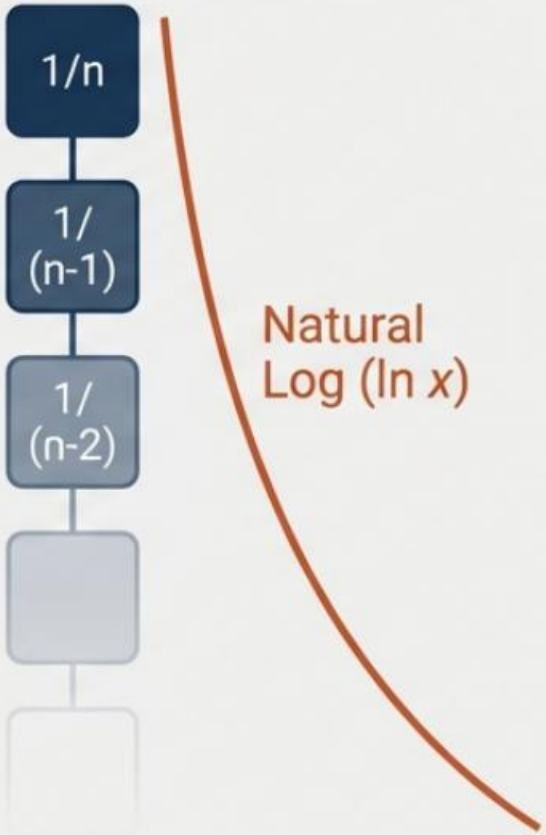
Applied Role

Context: Recursive algorithms processing inputs in pairs.

The Harmonic Chain

Species: Harmonic Series

$$T(n) = T(n-1) + 1/n$$



Key Insight

The Harmonic Series grows extremely slowly. Even for massive inputs, the accumulated cost remains small.

Applied Role

Context: Randomized algorithms, such as calculating the expected value in random permutations.

EXHIBIT LABEL

- **Series:** Sum of $1/i$
- **Approximation:** Natural Log ($\ln n$)
- **Complexity:** $\Theta(\log n)$

The Exponential Explosion

Species: Dual Recursion (Fibonacci)

$$T(n) = T(n-1) + T(n-2) + 1$$

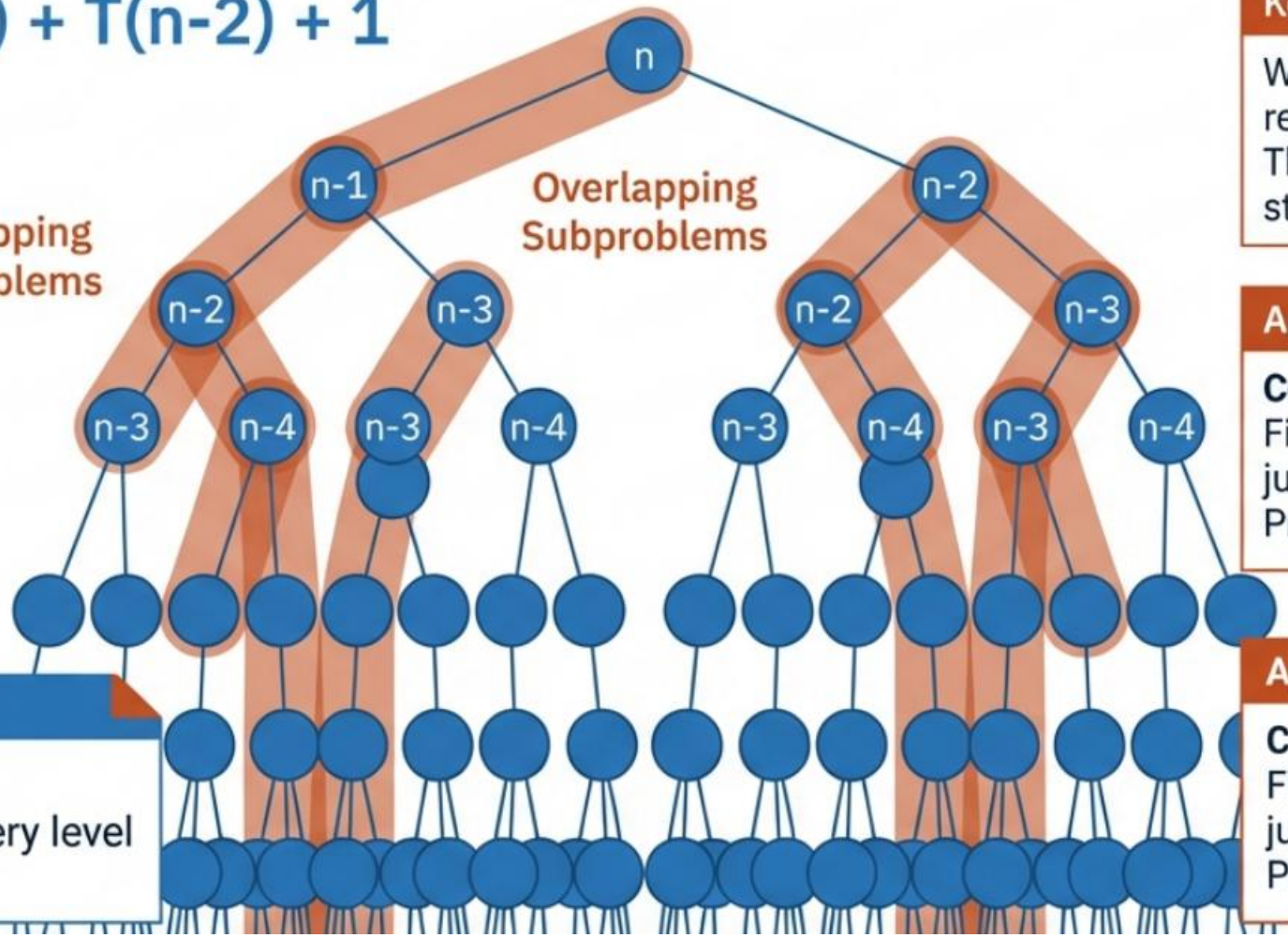


EXHIBIT LABEL

- **Tree Depth:** n
- **Growth:** Doubles every level
- **Complexity:** $\Theta(2^n)$

Key Insight

Without memoization, the redundancy is catastrophic. The work doubles at every step of depth n.

Applied Role

Context: Naive Recursive Fibonacci—the classic justification for Dynamic Programming.

Applied Role

Context: Naive Recursive Fibonacci—the classic justification for Dynamic Programming.

The Substitution Method

A Visual Derivation of O(n log n)

$$T(n) = 2T(n/2) + n$$

ALGORITHM ANALYSIS: SOLVING RECURRENCE VIA "PLUG & CHUG"

$$T(n) = 2T(n/2) + n$$

What is T(n/2)?

Replace n with n/2:

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2}$$

Iteration 1

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n$$

Simplify

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

Step 1 $$T(n) = 2^{1}T(n/2^{1}) + 1n$$

Step 2 $$T(n) = 2^{2}T(n/2^{2}) + 2n$$

Step 3 $$T(n) = 2^{3}T(n/2^{3}) + 3n$$

Step k (General Equation)

$$T(n) = 2^{k}T(n/2^{k}) + k \cdot n$$

We need to find the specific k where the recursion stops.

Recursion stops when input size = 1 (Base Case)

$$\frac{n}{2^k} = 1 \longrightarrow 2^k = n \longrightarrow \boxed{k = \log_2 n}$$

$$T(n) = \overset{\log_2 n}{2^k}\, T\left(\frac{n}{2^k}\right) + \overset{\log_2 n}{k} \cdot n$$

$$\downarrow$$

$$T(n) = n \cdot T(1) + n \cdot \log_2 n$$

$$= n + n \log n \quad \text{(Since T(1) is constant)}$$

COMPLEXITY CLASS

$$O(n \log n)$$

Questions

TASK: SOLVE USING THE TREE METHOD

01. $T(n) = 2T(n/2) + 10$

02. $T(n) = 3T(n/3) + n^2$

03. $T(n) = T(n-1) + T(n-2) + 1$

04. $T(n) = T(n-1) + n$

05. $T(n) = 3T(n/2) + n$

06. $T(n) = T(n/4) + T(n/2) + n^2$

07. $T(n) = T(n/3) + T(2n/3) + n$

08. $T(n) = 2T(n/4) + n/2$

09. $T(n) = 3T(n/4) + n$

10. $T(n) = T(n-1) + 1$

11. $T(n) = T(n-1) + 1/n$

12. $T(n) = T(n-1) + (n-1)^2$

Solve using substitution method

$$T(n) = T(n-1) + 1$$

$$T(n) = 2T(n/4) + \sqrt{n}$$